

Module 15 — Spring Boot & Microservices Company Interviews

A curated question bank grouped by type and by the companies that ask them. Use it as a rapid self-test: cover the answer, respond aloud, then verify against the referenced module. Big-tech Java/backend loops mix **fundamentals + system design + production debugging + behavioral**.

15.1 By Company (signature focus areas)

Company	Typical emphasis
Google	strong CS fundamentals, concurrency, complexity, design at scale, “why” behind abstractions
Amazon	Leadership Principles + scalability, idempotency, resilience, cost, ownership; bar-raiser
Microsoft	OOP/design, .NET-parallels, API design, testing, clean code
Uber	high-throughput, low-latency, geo/real-time, Kafka, sharding, consistency
Netflix	resilience (they birthed Hystrix/chaos), microservices, observability, JVM tuning
LinkedIn	Kafka (they created it), data infra, caching, feed/graph scale
VMware / Broadcom	deep Spring internals (Spring source), Tanzu, JVM, cloud-native
Oracle	JPA/Hibernate, SQL, transactions, JVM/GC internals
JPMorgan / Goldman Sachs	correctness, transactions/consistency, security, low-latency, resiliency, testing
Walmart Global Tech	microservices at retail scale, Kafka, caching, peak-traffic (Black Friday)
Adobe	API design, performance, cloud services, clean architecture
Atlassian	pragmatic system design, REST API design, multi-tenancy, testing, trade-offs

15.2 Conceptual Questions (rapid fire)

1. IoC vs DI; why constructor injection? (M2)
2. Full bean lifecycle; where are AOP proxies created? (M2)
3. How does auto-configuration work; how to override a bean? (M3)
4. DispatcherServlet request lifecycle. (M4)
5. Persistence context, dirty checking, flush vs commit. (M5)
6. N+1 problem — detect and fix. (M5)
7. `@Transactional` internals; which exceptions roll back; propagation. (M5)
8. Security filter chain & authentication flow. (M6)
9. Session vs JWT; OAuth2 vs OIDC. (M6)
10. Kafka delivery guarantees; ordering; consumer groups. (M8)
11. Caching strategies; safe invalidation. (M9)
12. Isolation levels & anomalies; optimistic vs pessimistic locking. (M5, M10)
13. G1 vs ZGC; JVM memory areas. (M1)
14. `CompletableFuture` composition; thread-pool sizing. (M1)
15. HikariCP sizing; pool exhaustion causes. (M10, M13)

15.3 Scenario-Based Questions

1. “Design an idempotent payment API that tolerates client retries.” → idempotency key + unique constraint / dedup store; at-least-once safe; 409 on replay with same result. (M7, M14)
2. “Order spans inventory + payment services — ensure consistency.” → orchestration saga + compensations + outbox + idempotent consumers. (M7)
3. “Handle a 10x Black-Friday traffic spike.” → autoscale (HPA), cache hot data, circuit breakers + timeouts + bulkheads, async where possible, DB read replicas, load test first. (M7, M9, M12)
4. “Guarantee per-user event ordering at high throughput.” → Kafka key by user → same partition. (M8)
5. “Migrate a monolith to microservices.” → strangler-fig, extract bounded contexts, DB-per-service via expand/contract, anti-corruption layer. (M7, M10)
6. “Run a job exactly once across N instances.” → distributed lock (Redis **SET NX PX**/Redisson) or leader election. (M9)

15.4 Production Issue Questions

1. Memory keeps growing until OOM — diagnose. (M13.2)
2. p99 latency spiked after a deploy — localize. (M13.4)
3. **Connection is not available** under load. (M13.6)
4. Kafka consumer lag growing. (M13.7)
5. CPU pegged at 100%. (M13.3)
6. Two threads hung (deadlock). (M13.5)
7. Cache hit ratio dropped, DB overloaded. (M13.8)
8. App won't start (bean/circular dependency error). (M13.11–12)

15.5 Architecture Questions

1. Design a URL shortener / rate limiter / notification service (apply caching, sharding, idempotency, async). (M7–M10)
2. How do you achieve zero-downtime deploys? (*rolling update + readiness + backward-compatible migrations* — M10, M12)
3. API gateway vs service mesh vs load balancer. (M7)
4. How do you secure service-to-service communication? (*mTLS / OAuth2 client-credentials / JWT resource server* — M6, M7)
5. Multi-tenancy strategies (DB-per-tenant vs shared schema + tenant id). (M10)
6. How do you make a service observable? (*metrics+logs+traces, RED/USE, SLOs* — M11)

15.6 Debugging Questions

1. Which tools capture thread state, heap, GC, CPU? (*jstack, jmap/MAT, GC logs, async-profiler/JFR, Actuator* — M13)
2. What do you capture before restarting a sick JVM? (*thread + heap dump, GC log.*)
3. How do you find which service causes a latency spike? (*distributed trace* — M11)
4. How do you detect a Java deadlock vs thread starvation? (*jstack deadlock section vs pool saturation/blocked-on-I/O* — M13)

15.7 Coding Questions

1. RESTful CRUD with pagination, validation, error handling. (M14)
2. Streaming file upload with validation. (M14)
3. Retry with exponential backoff + jitter and fallback. (M14, M7)
4. `@Cacheable` with stampede protection and eviction on write. (M14, M9)
5. Core Java: group/aggregate with Streams; sort by multiple fields; thread-safe counter; produce/merge parallel `CompletableFutures`. (M1)
6. Implement a token-bucket rate limiter (in-memory or Redis). (M9)

15.8 Follow-up Questions (interviewers dig deeper)

- “You said constructor injection — how does Spring know which constructor?” (*single constructor auto; else `@Autowired`.*)
- “You used JWT — how do you revoke one?” (*short TTL + refresh rotation + denylist.*)
- “You added an index — why might the planner ignore it?” (*low selectivity, stale stats, non-SARGable predicate, small table.*)
- “Retry — what if the operation isn’t idempotent?” (*don’t retry, or add an idempotency key.*)
- “Circuit breaker OPEN — what does the user see?” (*fast failure + fallback.*)
- “You cached it — how do you keep it consistent?” (*evict on write, TTL, accept eventual consistency, or write-through.*)
- “Saga failed midway — how do you recover?” (*compensating transactions; idempotent + retrievable steps; monitor stuck sagas.*)

15.9 Behavioral / System-design framing (senior signal)

- **STAR** for behavioral (Situation, Task, Action, Result); quantify impact.
- Amazon: map stories to **Leadership Principles** (Ownership, Bias for Action, Dive Deep, Deliver Results).
- For design: **clarify requirements** → **estimate scale (QPS, data)** → **API** → **data model** → **components** → **bottlenecks** → **trade-offs** → **failure modes**. State assumptions and trade-offs explicitly; there is no single right answer.
- Always end an answer with trade-offs and how you’d validate/monitor it.

Module 15 — Final Prep Cheat Sheet

Bucket	Nail these
Spring internals	bean lifecycle, proxies/self-invocation, auto-config, DispatcherServlet
Data	persistence context, N+1, @Transactional propagation/rollback, isolation, locking
Security	filter chain, AuthN flow, JWT vs session, OAuth2/OIDC, CSRF/CORS
Distributed	saga, outbox, idempotency, circuit breaker/retry/timeout/bulkhead
Messaging	partitions/ordering, consumer groups, delivery guarantees, DLQ
Caching	strategies, invalidation, stampede, distributed lock
JVM	memory areas, GC (G1/ZGC), thread pools, CompletableFuture
Ops	HikariCP, probes, observability (metrics/logs/traces), K8s objects
Debugging	symptom → evidence → isolate → fix → prevent; jstack/jmap/traces

Module 15 — Mock Interview (mixed loop)

1. (Concept) “Why does `@Transactional` on a private method do nothing?” → proxy-based AOP only advises public methods called through the proxy.
2. (Scenario) “Design an idempotent ‘create payment’ endpoint.” → idempotency key + unique constraint; return prior result on replay.
3. (Production) “Latency spiked after deploy; DB CPU normal.” → trace shows a downstream call without timeout; add timeout + circuit breaker; consider the new code path/N+1.
4. (Architecture) “Zero-downtime schema change on a huge table.” → expand/contract, backward-compatible, batched backfill, rolling deploy.
5. (Coding) “Rate-limit 100 req/min/user across instances.” → Redis token bucket (atomic), 429 + `Retry-After`.
6. (Follow-up) “How would you prove your fix worked?” → metrics/SLO dashboards before/after, load test, add an alert + regression test.

You’ve completed all 15 modules.

Review the **Master PDF** for the full curriculum, then drill the per-module cheat sheets and mock interviews until you can answer each aloud without notes. Good luck — you’re interview-ready.